COUNSEL: Modular MLOps Transformation Plan

Decoupled Plug-and-Play Policy Adapter, INT8/Sparse Compression, SRE Observability & Governance

Course: DS5619 Machine Learning Systems Operations | Timeline: 7-Day Sprint | Team: 4 Members (Equal Ownership)

1. Executive Summary, Instructor Directives & Codebase Audit Fixes

COUNSEL (Hegde et al., IEEE/ACM CCGrid 2023) provisions cloud VM instances across multi-component microservice chains using a Graph Convolutional Actor-Critic (GCN-PPO/A3C) agent. In the original repository, the simulation environment (`CloudEnv` in `model/env.py`) is tightly coupled to PyTorch tensors, hardcoded paths (`a3c/` vs. `pyt_save/`), and legacy Python 2 relative imports. This master plan refactors COUNSEL into a modular, contract-enforced MLOps platform that satisfies our instructor's core directives while integrating **every concept taught across Weeks 1-12 of DS5619**.

Instructor Directives & Architectural Alignment:
• **"Don't make it too complicated, just do plug-in plug-out models" (Week 9 Information-Hiding Contract):** Decouple `CloudEnv` from any single neural architecture via an abstract `BaseScalingPolicy` interface. Swap FP32 DRL, Compressed INT8/Pruned DRL, Reactive Heuristics, Static FinOps baselines, and External REST/LLM engines via `configs/config.yaml` with zero environment changes.
• **"Do small but do perfect" (Week 7 Three Ways & Automated Gates):** Avoid fragile 14-stage cloud pipelines. Build concise, self-contained Python modules (~40-60 lines each) backed by 100% deterministic `pytest` contract gates, a 3-job GitHub Actions workflow, and reproducible benchmarks.
• **"Complete Student Ownership & Viva Readiness":** Every team member owns a clean vertical slice and can defend both the COUNSEL code patches (`Setup.md`) and the exact mathematical formulas from the DS5619 lecture slides.

Critical Technical & Syllabus Upgrades Over Initial Draft Plan:
1. **Codebase Bug Fixes (`Setup.md` Verified):** Patches Python 2 relative imports in `chain.py`/`component.py`, fixes the 7-tuple unpacking crash in `test_policy.py` (`o, m, r, d, *_ = env.step(a)`), unifies `a3c/` and `pyt_save/` checkpoint paths, and replaces unsafe pickle `torch.load()` with hash-verified safe weight loading (Week 5 & Week 10).
2. **Complete Graph + Telemetry Contract (`SystemState`):** Adds `graph_obs` (spectral adjacency + node feature matrix) with Avro-compliant defaults alongside scalar SRE metrics so both `GCNActorCritic` and heuristic/API models operate strictly from `SystemState`.
3. **Added Missing Syllabus Pillars (Weeks 4, 6, 9, 10 & 11):** Integrates **Model Compression (Week 11 INT8 Quantization + Sparse COO Pruning + Roofline/Energy)** as a plug-in policy, **Security/Fairness/SHAP-LIME Explainability (Week 10)**, **Kahn's DAG Orchestrator + Exponential Backoff (Week 9)**, and **FTI Feature View Lineage (Week 4)**.

2. Two-Track Execution Strategy (Deterministic Local Core + Authentic Compute)

Track 1: Local MLOps Core (100% Deterministic & Fast)	Track 2: Authentic Training Run & Compute Telemetry
• Runs locally with `loadserver` Docker container (`network_mode: host`) in **under 8 minutes**. • Evaluates verified pre-trained checkpoints (`model1000.pt`) + compressed INT8/Pruned plugins across 20 episodes. • Executes full `pytest` contract gates, PSI drift harness, FGSM perturbation audit, and SHAP/LIME attributions. • Guarantees a complete, zero-risk project even if cluster queues are congested.	• Executes containerized training (`traina3c.py` / `train_cluster.sh`) on Madhava H100 node (or verified 29h local `std` run per `Setup.md`). • Logs step/epoch telemetry (Actor/Critic loss, utilization, SLO breaches, reward) to JSON/CSV & MLflow. • Captures PyTorch Tensor Profiler output (`profiler_output.txt`) & Roofline arithmetic intensity. • Fulfills Capstone Deliverables 1, 3 & 4 (convergence plot, live training view, compute/energy budget).

3. FTI Feature View & Formal Data Contracts (schema.py — Weeks 2, 3, 4 & 10)

To eliminate offline/online feature skew (Week 4 FTI pattern) and prevent Data Cascades (Week 3), `CloudEnv.get_contract_state()` acts as a unified **Feature View**. Schema evolution follows **Apache Avro's Rule** (Week 3 Slide 9: new/optional fields carry explicit defaults for backward/forward compatibility):

Contract Schema	Pydantic Field Specification & Types (schema.py)	MLOps Role & Lecture Concept Operationalized

SystemState (Input Feature View)	`schema_version: str = '1.0'`
	`step: int, event_time_s: float, ingestion_time_s: float`
	`arrival_rate: float, slo_latency: float, current_latency: float`
	`budget_remaining_cpu: float, budget_remaining_mem: float`
	`cpu_utilizations: List[float], mem_utilizations: List[float]`
	`component_utils: List[float], target_component: int`
	`scale_direction: int (1=Up, 0=Down)`
	`valid_action_mask: List[int] (len=nconf, mask[0]==1)`
	`graph_obs: Optional[List[List[float]]] = None`
		• **Data Contract Gate (Week 3):** Validates units/ranges (`0<=util<=1`, `slo>0`) at ingestion boundary.
		• **Types of Time (Week 3):** Distinguishes `event_time_s` vs. `ingestion_time_s`.
		• **Avro Schema Evolution (Week 3):** Defaulted `graph_obs` supports both GCN-PPO and rule/API models seamlessly.
		• **Point-in-Time Correctness (Week 4):** Prevents future trace leakage.

ScalingAction (Output Contract)	`action_id: int (0..nconf-1, 0=No-Op)`
	`model_name: str, model_version: str = 'v1.0'`
	`inference_latency_ms: float`
	`was_masked_fallback: bool = False`
	`feature_attributions: Optional[Dict[str, float]] = None`
	`reasoning: Optional[str] = None`
		• **Action Allowlist Guardrail (Week 10 Slide 12):** Blocks unmasked/injected actions (`mask[a]==0`) and falls back to No-Op (`0`).
		• **Anti-Extraction API (Week 10 Slide 5):** Exposes discrete `action_id` without leaking raw neural logits.
		• **SHAP/LIME Explainability (Week 10 Slides 18-19):** Records per-feature attribution weights in `feature_attributions`.

4. The Plug-and-Play Policy Suite (BaseScalingPolicy.select_action)

Every scaling model implements `BaseScalingPolicy.select_action(state: SystemState) -> ScalingAction` (Week 9 Slide 11: *"The Deployment API Is a Contract, Not Just an Endpoint"*). By including **Compressed INT8 & Sparse-Pruned GCN-PPO models (Week 11)** directly inside the plugin suite, our benchmark evaluates RL, Model Compression, Reactive Control Loops, FinOps Bounds, and Async External APIs under one harness:

Plug-in Policy	Category & Source	Mechanism & Mathematical Control Logic	Syllabus Role & Evaluation Purpose
1. GCN-PPO / A3C (FP32 Baseline)	DRL Teacher (`baseline_fp32`)	Passes `state.graph_obs` + `valid_action_mask` into 2-layer `GCNActorCritic` (4 bytes/param FP32 checkpoint `model1000.pt`). Applies `-Inf` logit masking before sampling.	Primary CCGrid 2023 paper baseline (Zenodo / 29h local run / Madhava H100 cluster checkpoint).
2. Compressed DRL (INT8 + Sparse COO)	**NEW: Week 11** Compression Plugin	Implements Week 11 Lab 11 variants on `GCNActorCritic`: • **Symmetric INT8 PTQ:** `s=max(|W|)/127, q=clamp(round(W/s),-127,127)` (4× weight size reduction). • **Magnitude Pruning (70%):** Compares `pruned_dense_fp32` (0B saved) vs. `pruned_sparse_coo` (value+index).	Operationalizes **Week 11 Model Compression**, Roofline memory-bandwidth analysis, & Horowitz DRAM energy savings (21,333× vs 1×).
3. Rule-Based Reactive Autoscaler	Heuristic Control (Week 9 Reactive)	Implements Week 9 Slide 13 reactive laws: Scale UP if `util > 0.80` or latency ratio `ceil(lat/slo) > 1`; Scale DOWN if `util < 0.25`; else No-Op (`0`). Respects `Component.check_TTL()` cooldown.	Primary deterministic extension: exposes reactive cold-start lag and threshold oscillation vs. predictive GCN-PPO.
4. Static FinOps Baselines (OP / UP)	FinOps Bounds (Week 9 Slide 14)	• **Over-Provisioned (OP, knob=0.03):** Selects max valid VM flavor (pre-provisioned warm headroom). • **Under-Provisioned (UP, knob=0.07):** Selects min flavor / No-Op (cost-optimized reactive floor).	Establishes the Week 9 FinOps Pareto bounds: maximum cloud spend ceiling (0% cold-start risk) vs. SLO breach floor.
5. External API / LLM Async Adapter	Async REST Plugin (Weeks 6, 9 & 10)	Serializes `SystemState` to JSON. Uses **Week 6 Mercari Async / Cache Pattern** + **Week 9 Exponential Backoff** (`delay = base * 2^attempt`) + **Week 10 Action Allowlist** fallback to No-Op (`0`) on timeout/invalid action.	Proves true language-independent extensibility, async latency decoupling, and prompt-injection/allowlist safety.

5. Team Structure & Equal 4-Member Workload Allocation

Responsibilities are partitioned into four orthogonal domains so each student writes ~150-200 lines of clean, modular Python code and can independently defend their subsystem and corresponding DS5619 lecture weeks during the viva:

Member & Domain	Syllabus Weeks Owned & Core Focus	Concrete Code Deliverables & Viva Defense Artifacts
Member 1 Environment, Feature View & Heuristics	**Weeks 2, 3, 4 & 9 (Reactive):** Data contracts, Avro schema evolution, FTI Feature View extraction, legacy codebase patches, and reactive/FinOps scaling laws.	• `schema.py`: Pydantic `SystemState` & `ScalingAction` (Avro defaults, Types of Time, range validators). • Patch legacy imports in `chain.py`/`component.py` & 7-tuple unpack in `test_policy.py`. • `CloudEnv.get_contract_state()` unified FTI Feature View hook in `model/env.py`. • `adapters/heuristic_adapter.py`: Reactive Threshold/Latency-Ratio autoscaler + OP/UP static bounds.
Member 2 DRL Adapters, Model Compression & Security Guardrails	**Weeks 6 (Async), 9 (API), 10 (Security) & 11 (Compression):** Polymorphic policy interface, FP32 & INT8/Pruned DRL adapters, Async REST/LLM adapter, and action allowlisting.	• `adapters/base_policy.py`: Abstract interface + Week 10 Action Allowlist guardrail & anti-extraction sanitizer. • `adapters/drl_adapter.py`: Safe weight loader for `GCNActorCritic` (`a3c/` & `pyt_save/`). • `adapters/compressed_drl_adapter.py`: Symmetric INT8 quantization (`s=max(|x|)/127`) + 70% Sparse COO & Dense pruning benchmark + Horowitz energy/Roofline calculator. • `adapters/external_api_adapter.py`: Mercari async response cache + exponential backoff retry.
Member 3 SRE Observability, PSI Drift, Fairness & Explainability	**Weeks 8 (SRE/Drift) & 10 (Responsible AI):** Unified benchmark runner, 4 Golden Signals, SLI/SLO Error Budget, 6-drift taxonomy & PSI, FGSM robustness, Fairness & SHAP/LIME.	• `evaluator.py`: 20-episode benchmark runner logging 4 Golden Signals (separating ok vs. breach latency) & Error Budget burn. • `monitoring/drift.py`: Feature & Prediction PSI (`<0.1, 0.1-0.25, >=0.25`) + Hierarchical Alerting deduplicator. • `monitoring/security_fairness.py`: FGSM-style telemetry perturbation audit (`eps=0..0.2`) + Per-Component Fairness (`MetricFrame` parity) + LIME/Shapley local attribution explainer. • `plotters/compare_models.py`: Multi-objective Pareto frontier visualizer (Cost vs. SLO Breach vs. Size/Latency).
Member 4 Containers, Kahn's DAG, CI/CD Gates, Registry & Cluster Run	**Weeks 2 (YAML), 5 (Registry), 6 (Docker), 7 (CI/CD/GitOps) & 9 (DAG/Slurm):** Norway-safe config, cgroup containers, Kahn's DAG orchestrator, 2-tier gates, RBAC Model Registry & H100 run.	• `configs/config.yaml` (quoted scalars avoiding Norway Problem) + `docker-compose.yaml` with Linux `cgroup` limits. • `orchestrator/dag.py`: Kahn's topological sort (`topological_order()`) + retry runner executing the benchmark DAG. • `.github/workflows/ci.yml` (3-job Code Gate: `lint`, `unit-test`, `integration-test`) + `registry/release_gate.py` (5-part Model Release Gate). • `registry/` with RBAC (`data_owner` vs `data_scientist`), `generate_model_card.py`, & Compute Budget Report.

6. Detailed 7-Day Synchronized Sprint Execution Matrix

A dependency-ordered daily schedule ensuring all four members build, unit-test, and integrate their components without blocking dependencies:

Day	Member 1 (Env / FTI / Heuristics)	Member 2 (DRL / Compression / API)	Member 3 (SRE / Drift / Responsible AI)	Member 4 (Config / DAG / CI / Registry)
Day 1 Patches & Setup	Apply `Setup.md` patches in `chain.py`, `component.py`, and `test_policy.py`. Verify `loadserver` container.	Audit `GCNActorCritic.step()` in `a3c.py` & `core.py`. Design `BaseScalingPolicy` contract.	Deconstruct `inference.py`. Define unified telemetry schema for Golden Signals & Error Budget.	Configure `docker-compose.yaml` with `network_mode: host` & `cgroup` CPU/RAM limits. Init Git branch protection.
Day 2 Contracts & Config	Implement `schema.py` (`SystemState` & `ScalingAction`) with Avro defaults, Types of Time & range checks.	Implement `base_policy.py` with Action Allowlist guardrail & `drl_adapter.py` loading `model1000.pt`.	Build `evaluator.py` runner executing 20-episode traces against any polymorphic `BaseScalingPolicy`.	Create `configs/config.yaml` (quoted scalars for Norway Problem) & hash-locked `requirements.lock`.
Day 3 Core Plugins & DAG	Add `get_contract_state()` FTI Feature View hook to `CloudEnv` in `model/env.py`.	Build `compressed_drl_adapter.py`: Symmetric INT8 PTQ (`s=max(|x|)/127`) + 70% Sparse COO & Dense pruning.	Implement Golden Signals logger (separating non-breach vs. breach P50/P99 latency) & SLO Error Budget tracker.	Implement `orchestrator/dag.py` using **Kahn's Algorithm** (`topological_order` + cycle check + backoff retry).
Day 4 Policies & Drift	Implement `heuristic_adapter.py`: Week 9 Reactive Threshold (80%/25%) + Latency-Ratio control + TTL check.	Implement `external_api_adapter.py` with Mercari async response cache & exponential backoff (`2^attempt`).	Build `monitoring/drift.py` computing Feature & Prediction **PSI** (`0.1 / 0.25`) + Hierarchical Alerting.	Launch Track 2 training (`train_cluster.sh` / `traina3c.py`). Build `registry/rbac.py` (`data_owner` vs `data_scientist`).
Day 5 RAI, Compression & CI	Implement Static FinOps Over-Provisioned (op) & Under-Provisioned (up) baselines. Smoke test `f5-c3`.	Run Week 11 Compression Lab benchmark (`fp32`, `pruned_dense`, `pruned_coo`, `int8`) + Roofline/Horowitz energy report.	Build `monitoring/security_fairness.py`: FGSM perturbation audit (`eps=0..0.2`), `MetricFrame` fairness, & LIME/SHAP explainer.	Build `.github/workflows/ci.yml` (Jobs: `lint`, `unit-test`, `integration-test`) + `registry/release_gate.py`.
Day 6 Full Benchmark & Gates	Write `tests/test_env_contract.py` verifying schema evolution, Avro defaults, and boundary validation.	Write `tests/test_adapters.py` verifying all 5 policy families obey `valid_action_mask` & INT8 accuracy.	Execute unified 20-episode benchmark across all 5 policies. Generate Pareto frontier & drift plots.	Run `generate_model_card.py` producing `MODEL_CARD.md` + Lineage graph (`Data->FeatureView->Model->Deploy`).
Day 7 Audit & Viva Prep	Verify end-to-end deterministic execution (<8 mins on CPU). Freeze `schema.py` & `heuristic_adapter.py`.	Compile Compression & Latency table (FP32 vs INT8 vs Sparse COO vs Rule vs Async API).	Finalize SRE Golden Signals, PSI Drift, FGSM Robustness, and SHAP/LIME attribution charts.	Compile Compute & Energy Budget Report, verify all 4 Capstone Rubric deliverables, and rehearse viva.

End-to-End Closed-Loop MLOps Data & Control Flow (Weeks 3, 4, 7, 8, 9, 10 & 11):
1. **Ingestion & Feature View (Weeks 3-4):** `loadserver` streams workload traces (`load_*.npy`) -> `CloudEnv.get_contract_state()` extracts point-in-time `SystemState` validated by Pydantic/Avro rules.
2. **Plug-and-Play Policy Execution (Weeks 6, 9, 10, 11):** `BaseScalingPolicy.select_action(state)` dispatches to configured plugin (`DRL-FP32`, `DRL-INT8/Sparse`, `Reactive-Rule`, `Static-OP/UP`, or `Async-API`), enforces the **Action Allowlist Guardrail** (`valid_action_mask`), and attaches **LIME/Shapley feature attributions**.
3. **SRE Observability & Closed-Loop Governance (Weeks 5, 7, 8, 10):** `evaluator.py` & `drift.py` compute 4 Golden Signals, SLI/SLO Error Budget, Feature/Prediction **PSI**, and `MetricFrame` component fairness -> verified by the **5-Part Model Release Gate** and documented in `MODEL_CARD.md`.

7. Complete DS5619 Lecture Syllabus Mapped to COUNSEL Implementation

Every single slide deck in `MLOPS_SLIDES/` is explicitly operationalized in our repository with concrete mathematical formulas and code files:

DS5619 Lecture Deck	Core Concepts, Formulas & Warnings in Slides	Direct COUNSEL Implementation & Verification File
Week 2-3: Data Formats & Config Management	Row (`ndarray`) vs Column (`DataFrame`) layout; Config Debt & 6 Principles (Huyen); YAML syntax & **The Norway Problem** (barewords `NO/off/on` parsed as booleans); TOML vs YAML.	`configs/config.yaml` quotes all string scalars; validated via `yaml.safe_load()` + Pydantic config model, replacing hardcoded parameters in `train.py` & `inference.py`.
Week 3: Sources, ETL/ELT, Schema Evolution & Validation	Dataset compilation (schema validation = type checker); **Types of Time** (Event, Ingestion, Processing); **Apache Avro's Rule** (add/remove fields only with defaults); **Data Cascades**; **Data Contracts**; Great Expectations.	`schema.py` enforces `SystemState` & `ScalingAction` contracts at the ingestion boundary, tracks `event_time_s` vs `ingestion_time_s`, and uses Avro defaults for `graph_obs`.
Week 4: Lifecycle, Versioning, Feature Stores & Lineage	**Code (Git) + Data (DVC) + Model (Registry)** separation of concerns; **FTI Architecture**; Feature Groups -> Feature Views; **Point-in-Time Correctness** (no future leakage / skew); **Lineage & DODD**.	`CloudEnv.get_contract_state()` provides one shared Feature View for training & inference; `registry/lineage.json` traces `load_*.npy` SHA-256 hashes -> Feature View -> Model -> Deployment.
Week 5: Model Registry & Governance (Model Cards)	"Training is compiling; Registry is the Artifact Store"; File formats (`.pt`, `.onnx`, `.safetensors`) & `.pkl` **arbitrary code execution risk**; Hyperparameter runs; **6-Section Model Cards**.	`registry/` stores immutable checkpoints with SHA-256 hashes (replacing unsafe pickle loads); `generate_model_card.py` auto-builds `MODEL_CARD.md` with embedded PNG plots.
Week 6: Containerization, cgroups & Async Execution	Virtualization levels; **Linux Namespaces** (visibility isolation) + **Linux cgroups** (CPU/RAM limits) = Docker; **Mercari Asynchronous Execution Pattern** (`Client -> Queue/Cache -> Predictor`, FIFO/Cache triggers, retry loops).	`docker-compose.yaml` isolates `loadserver` & `counsel` with explicit `cgroup` CPU/memory bounds; `external_api_adapter.py` uses async state-hash caching to prevent simulator blocking.
Week 7: CI/CD, Integration Testing, Gates & GitOps	Three Ways & DORA 4 metrics; **CI + CT + CD Closed Loop**; 3-Job GitHub Actions DAG (`lint` + `unit-test` -> `needs` -> `integration-test`); **Code Gate vs. 5-Part Model Release Gate**; **GitOps** (IaC + MRs + DTAP).	`.github/workflows/ci.yml` runs `flake8`, `pytest`, and container health/inference integration test; `registry/release_gate.py` certifies the 5-part Evidence Bundle before promotion.
Week 8: Observability, SRE Golden Signals & PSI Drift	**SLI, SLO, SLA & Error Budget**; **4 Golden Signals** (separate good vs failed latency!); White-box vs Black-box; **6 Drift Types**; **PSI Formula** (`sum((A-E)*ln(A/E))`, `0.1/0.25`) & Napkin Math 12.9; **Alert Fatigue** (`1-(1-alpha)^N`) & Hierarchical Alerting.	`evaluator.py` tracks 4 Golden Signals & Error Budget burn; `monitoring/drift.py` logs raw features to compute Feature/Prediction PSI & KPI degradation with Hierarchical Alerting.
Week 9: Orchestration (Kahn's DAG), Serving & Reactive Scaling	**Kahn's Topological Sort (`topological_order`)** & **Exponential Backoff (`base*2^attempt`)**; Slurm (Imperative) vs K8s (Declarative); **Deployment API as Information-Hiding Contract**; GPU Cold Start (420s); **3 Reactive Scaling Laws** & **FinOps Trade-off**.	`orchestrator/dag.py` runs Kahn's DAG with exponential backoff; `base_policy.py` enforces the serving contract; `heuristic_adapter.py` implements reactive threshold/latency scaling vs OP/UP FinOps bounds.
Week 10: Security, Privacy, RBAC, Fairness & SHAP/LIME	Poisoning, **Model Extraction**, & **FGSM Adversarial Examples** (`clip(X+eps*sign)`); `requirements.lock` (`torchtriton`); **Hopsworks 2-Role RBAC**; **Agent Action Allowlisting**; **Fairlearn** `MetricFrame`; **SHAP (`phi_i`) & LIME**.	`base_policy.py` enforces `valid_action_mask` allowlisting & hides logits; `security_fairness.py` runs FGSM telemetry audits, per-component fairness, and LIME/Shapley attributions; `rbac.py` gates deployments.
Week 11: Model Compression, Roofline & Energy Hierarchy	**Unstructured vs Structured Pruning** (`pruned_dense_fp32` [0B saved] vs `pruned_sparse_coo`); **Symmetric INT8 PTQ** (`s=max(|x|)/127`, 4× smaller); Distillation; **Roofline Model** & **Horowitz Energy** (DRAM read 21,333× vs INT8 add 1×).	`adapters/compressed_drl_adapter.py` plugs `quantized_int8` and `pruned_sparse_coo` `GCNActorCritic` variants into the benchmark, comparing size (KB), agreement %, latency, and DRAM energy.

8. Quantitative Benchmark Design & Compression / Security Audit Matrices

To make our evaluation airtight for the viva and final report, `evaluator.py` and `compressed_drl_adapter.py` automatically generate three linked benchmark tables across identical 20-episode workload traces (`nconf=5, ncomp=3, slo=150ms`):

Benchmark Dimension	Metrics Logged per Policy Plugin	Course Theory Demonstrated
A. Multi-Policy SRE & FinOps Leaderboard (All 5 Policy Families)	• **Latency (Golden Signal 1):** P50 / P99 latency (ms) split by SLO-compliant vs. breached steps + Decision Latency (ms). • **Traffic & Saturation (Signals 2 & 4):** Mean arrival rate (req/s) & component CPU/RAM utilization (%). • **Errors & FinOps Cost (Signal 3):** SLO Breach Rate (%), Error Budget Burn (%), & Budget Overrun.	Proves Week 8 Four Golden Signals & Error Budget + Week 9 Reactive vs. Predictive DRL vs. Static FinOps Pareto trade-offs.
B. Week 11 Model Compression Table (`GCNActorCritic` Variants)	1. `baseline_fp32` (Unmodified FP32 weights, 4 bytes/param) 2. `pruned_dense_fp32` (70% magnitude-pruned, stored dense -> 0% disk savings) 3. `pruned_sparse_coo` (70% pruned, COO value + index format -> ~1.66× smaller) 4. `quantized_int8` (Symmetric INT8 `s=max(|x|)/127` -> **4.0× smaller**, >98% action agreement)	Reproduces **Week 11 Lab 11** directly on COUNSEL's GCN-PPO network, diagnosing Roofline memory-bound regime & Horowitz DRAM energy savings.
C. Week 10 Adversarial, Fairness & Explainability Audit	• **FGSM Perturbation Curve:** Policy reward & SLO breach stability under `eps in {0.0, 0.05, 0.10, 0.20}`. • **Per-Component Fairness (`MetricFrame`):** Equal Opportunity (SLO compliance rate across components `C0, C1, C2`). • **SHAP / LIME Attribution Check:** Verifies `baseline + sum(phi_i) = prediction` on scaling decisions.	Operationalizes **Week 10** inference-stage adversarial testing, Fairlearn group parity, and Regulation-B style decision explainability.

9. Capstone Rubric Deliverables Checklist & Repository File Map

Direct verification against the official DS5619 capstone grading rubric:

Official Capstone Deliverable	How Our COUNSEL MLOps Implementation Satisfies It	Verification Artifact in Repo
1. Reproduced Baseline Convergence Plot	Extracted from published `progress.csv` / `rewardsw00.json` and verified via authentic training run (`Setup.md` 29h CPU / Madhava H100 cluster).	`charts/baseline_convergence.png`
2. Implemented Modular Extension(s)	**Decoupled Plug-and-Play Policy Adapter Layer (`BaseScalingPolicy`)** comparing FP32 GCN-PPO, **Week 11 INT8/Sparse Compressed GCN-PPO**, **Week 9 Reactive Autoscaler**, Static FinOps bounds, and Async API adapter.	`adapters/* & charts/pareto_comparison.png`
3. Training & SRE Monitoring View	Epoch/step telemetry (Actor/Critic loss, reward, utilization, SLO breaches) + 4 Golden Signals, SLI/SLO Error Budget, Feature/Prediction PSI drift, and FGSM/Fairness audit.	`infer_logs/sre_metrics.csv & charts/psi_drift_report.png`
4. Documented Compute & Energy Budget	Documents CPU/GPU training hours, PyTorch Tensor Profiler (`profiler_output.txt`), Roofline arithmetic intensity, and Horowitz DRAM vs. INT8 inference energy savings.	`reports/compute_budget_report.md` `MODEL_CARD.md`

Clean Modular Repository Layout (All New Files ~40-60 Lines, 100% Viva-Defensible):

```
counsel/
|-- configs/config.yaml & requirements.lock # Week 2 Norway-safe YAML config & Week 10 hash-pinned deps
|-- schema.py # Week 3/4 Pydantic SystemState (FTI Feature View) & ScalingAction
|-- adapters/ # Week 9 Information-Hiding Plug-and-Play Policy Suite
| |-- base_policy.py # Abstract contract + Week 10 Action Allowlist & Anti-Extraction guardrails
| |-- drl_adapter.py # FP32 GCN-PPO / A3C wrapper (unified a3c/ & pyt_save/ safe loader)
| |-- compressed_drl_adapter.py # Week 11 Symmetric INT8 PTQ + 70% Sparse COO/Dense Pruning + Energy calc
| |-- heuristic_adapter.py # Week 9 Reactive Threshold/Latency-Ratio autoscaler + OP/UP FinOps bounds
| +-- external_api_adapter.py # Week 6 Mercari Async Cache + Week 9 Exponential Backoff REST/LLM wrapper
|-- orchestrator/dag.py # Week 9 Kahn's Topological Sort (topological_order) & Retry DAG runner
|-- evaluator.py # Week 8 Unified 20-episode runner logging 4 Golden Signals & Error Budget
|-- monitoring/
| |-- drift.py # Week 8 Feature/Prediction PSI (<0.1, 0.1-0.25, >=0.25) & Hierarchical Alerting
| +-- security_fairness.py # Week 10 FGSM perturbation audit, MetricFrame fairness & LIME/SHAP explainer
|-- registry/ # Week 5/7/10 Immutable checkpoints, RBAC, 5-part Release Gate & MODEL_CARD.md
|-- plotters/compare_models.py # Week 9/11 Pareto frontier, Compression tradeoff & Convergence visualizers
+-- .github/workflows/ci.yml & tests/ # Week 7 3-Job CI Pipeline (flake8, pytest contract gates, container integration)
```